# Efficient Fine Tuning and Inference Optimization for TinyBert

Garima Mahajan (gm3228), Preeti Mishra (pm3667)
New York University Tandon School of Engineering
github repository link

***Abstract*—This project focuses on enhancing the fine-tuning and inference optimization processes of TinyBERT, a lightweight variant of the BERT model, for more efficient deployment in real-world applications. The work involves conducting a comprehensive literature review, implementing fine-tuning techniques, and exploring various optimization strategies such as LoRA, Quantization, Flash Attention, Wandb sweeps. The findings aim to provide a practical framework for deploying TinyBERT in resource-constrained environments with minimal latency and optimal performance.**

***Index Terms*—TinyBERT, LoRA, Quantization, Flash Attention, Wandb.**

## I. Introduction

In recent years, transformer-based models such as BERT have become a cornerstone of natural language processing (NLP) tasks. However, their substantial computational requirements pose challenges for deployment in resource-constrained environments. TinyBERT, a compact version of BERT, offers a promising solution to this issue by maintaining comparable performance with a reduced computational footprint.

This project aims to further optimize TinyBERT by addressing two key areas: fine-tuning for specific downstream tasks and improving inference efficiency. The MRPC dataset, presents pairs of sentences extracted from online news sources and challenges the model to identify whether the sentences in each pair convey the same meaning. Various optimization techniques, quantization, LoRA adapters, FlashAttention, are explored to achieve the dual objectives of enhanced performance and reduced inference latency.

Preliminary experiments provide baseline metrics that guide the development of optimization strategies. The project's outcomes are expected to benefit applications requiring low-latency NLP models, such as conversational agents, edge devices, and mobile applications. This report outlines the project's progress, highlights its achievements, and discusses challenges and future directions.

### A. Background and Motivation

Transformer-based models like BERT have revolutionized natural language processing (NLP), demonstrating exceptional performance across a wide range of tasks. However, their deployment in real-world applications, particularly in resource-constrained environments, is hindered by their significant computational and memory requirements. TinyBERT, a distilled and efficient version of BERT, emerges as a promising alternative by maintaining high accuracy with reduced complexity. Despite these advantages, further optimization is necessary to enhance TinyBERT's usability in latency-sensitive scenarios, such as mobile applications and edge devices. Leveraging fine-tuning techniques and advanced optimization methods can unlock its potential, making it viable for real-time applications. This project explores these possibilities, motivated by the need for lightweight yet powerful NLP models in practical deployments.

### B. Problem Statement

While TinyBERT offers a promising solution for resource-limited scenarios, several challenges hinder its deployment in real-world applications:

**Fine-Tuning for MRPC**: The MRPC dataset, designed for paraphrase detection, demands fine-grained optimization of TinyBERT to achieve meaningful performance. This task involves addressing the inherent complexity and subtleties of identifying semantically equivalent sentences.
**Inference Optimization:** Current TinyBERT configurations may still incur significant latency and computational overhead during inference, necessitating advanced optimization techniques.
**Resource Limitations:** Limited access to high-performance computing resources poses challenges in efficiently experimenting with and deploying optimization strategies.

### C. Objectives and Scope

*1) Objective:* Fine-Tuning for MRPC: Apply advanced optimization techniques such as quantization, LoRA adapters and WnB sweeps to fine-tune TinyBERT for the MRPC task and enhance its accuracy.
Optimization for Inference: Implement methods like FlashAttention to improve inference speed and reduce latency while maintaining or improving model performance.
Performance Benchmarking: Evaluate the optimized TinyBERT model using metrics such as accuracy, latency and compare results against baseline metrics.

*2) Scope:* This paper is focused on optimizing TinyBERT model for paraphrase detection using the MRPC dataset. It involves implementing task-specific fine-tuning and evaluating the impact of various optimization strategies. The outcomes aim to provide a TinyBERT model with better accuracy and improved inference latency.

## II. Literature Review

### A. Review of Relevant Literature

This study builds upon a robust body of prior research, each contributing unique insights into optimizing and fine-tuning language models. The foundational work on LoRA (Low-Rank Adaptation) by Hu et al. [1] highlights efficient methods to adapt large language models with minimal computational overhead, offering a significant reduction in trainable parameters. This approach aligns with the goals of this project to improve model efficiency without compromising performance.

Donisch et al. [2] delve into inference optimization, discussing challenges and practical considerations when deploying large models. These insights informed the adoption of quantization strategies to achieve faster inference times. Similarly, the work of Dao et al. [3] on FlashAttention inspired the integration of memory-efficient attention mechanisms, crucial for reducing computational load during training and inference.

Rowan Zellers et al. [4] introduced the HellaSwag dataset, showcasing challenges in reasoning and contextual understanding. While initially considered for this project, the dataset's complexity exceeded TinyBERT's capabilities, prompting the shift to the MRPC dataset—a widely recognized benchmark for paraphrase detection as described by Wang et al. [6]. The decision reflects practical constraints while adhering to high evaluation standards.

Finally, the integration of bnb-4bit quantization [5] provided a compact yet effective model configuration, aligning with recent advancements in model compression and efficiency. These references collectively shaped the methodologies and innovations presented in this research.

### B. Identification of Gaps in Existing Research

While inference optimization techniques [2] are widely explored, the impact of combining multiple methods—such as LoRA, quantization, and FlashAttention—remains unexplored, especially for lightweight models.

Moreover, datasets like HellaSwag [4] test advanced reasoning capabilities but are not always practical for smaller models such as TinyBERT. The need for systematic guidelines on dataset selection for fine-tuning smaller models is a notable gap.

Addressing these gaps could improve the accessibility and efficiency of deploying lightweight models in real-world applications.

## III. Methodology

### A. Data Collection and Preprocessing:

Dataset Used: MRPC, The Microsoft Research Paraphrase Corpus (MRPC) was selected as the dataset for fine-tuning the TinyBERT model. MRPC is a binary classification task that involves determining whether two sentences in a pair are semantically equivalent. It is a widely used benchmark in NLP and is well-suited for evaluating the paraphrase detection capabilities of models like TinyBERT.

**Preprocessing Pipeline**:

Dataset Loading: The MRPC dataset was loaded using the Hugging Face datasets library. The dataset includes training, validation, and test splits, with sentence pairs and corresponding binary labels indicating whether the sentences are paraphrases.

Tokenization: The Hugging Face Tokenizer for the pre-trained model "`huawei-noah/TinyBERT_General_4L_312D`" was used. Sentences were tokenized with truncation and padding to ensure a uniform maximum length of 512 tokens per pair. The tokenizer outputs tensors compatible with PyTorch for seamless model integration.

Data Preparation: Batches were prepared for evaluation, ensuring the data was formatted to include tokenized inputs (`input_ids`, `attention_mask`, and `token_type_ids`).

Each batch was moved to the same device (CPU or GPU) as the model to optimize performance during inference.

### B. Model Selection:

The project utilizes TinyBERT, a distilled version of the BERT model, specifically the pre-trained model "`huawei-noah/TinyBERT_General_4L_312D`". This choice was motivated by the following factors:

1. Pre-Training Features: The selected model is a general-purpose TinyBERT that was pre-trained on a large corpus, making it a robust starting point for fine-tuning on specific downstream tasks, such as paraphrase detection with MRPC.
2. Task Adaptability: TinyBERT's architecture allows for flexibility in fine-tuning and optimization for specific tasks. Its smaller size does not compromise its ability to adapt to paraphrase detection, especially when combined with task-specific optimization techniques.
3. Hugging Face Integration: TinyBERT is fully compatible with the Hugging Face library, which offers pre-trained weights, tokenizers, and utilities for efficient fine-tuning. This compatibility simplifies implementation and accelerates experimentation.

### C. Optimization Procedure(s):

The selected model, TinyBERT (General 4L, 312D), is a lightweight Transformer-based architecture with four layers and a hidden size of 312. It employs 12 attention heads to capture relationships within the input data. The combination of these optimization techniques ensures the model performs effectively on the MRPC dataset while balancing accuracy, speed, and resource efficiency. It focuses on optimizing both training and inference processes to achieve efficient fine-tuning and enhanced deployment performance for the TinyBERT model. The optimization techniques employed include LoRA, Quantization, W&B Sweeps for raining and Flash Attention for inference.




Fig. 1. Training progress over steps

1. Low-Rank Adaptation (LoRA): LoRA is used to fine-tune the model efficiently by introducing low-rank adaptation matrices into selected layers while keeping the majority of the pre-trained weights frozen. This technique focuses on specific modules in the model, such as the query, key, and value projection layers of the Transformer. Key parameters include a rank of 16, a scaling factor of 16, and a dropout rate of 5%. By targeting these layers, the number of trainable parameters is significantly reduced, enabling faster training while maintaining performance.

2. Quantization: It reduces the precision of model weights and activations to 4-bit, thereby lowering memory usage and computational requirements. The implementation uses advanced techniques such as double quantization for enhanced precision and non-negative distribution for efficient quantization. All computations are performed in reduced precision (bfloat16) to further optimize inference speed. This approach is particularly effective for deploying models in resource-constrained environments without substantial loss in accuracy.

3. Hyperparameter Optimization Using Weights & Biases (W&B) Sweeps:

Hyperparameter optimization was carried out using a random search methodology facilitated by W&B sweeps. The sweep explored a wide range of configurations, including optimizers (such as, AdamW, Adafactor, SGD), learning rates between $1 \times 10^{-5}$ to $1 \times 10^{-3}$, and various learning rate schedulers like linear, cosine, and constant with warmup. Additionally, it fine-tuned parameters such as weight decay, warmup ratio, batch size, and gradient accumulation steps. LoRA-specific parameters, such as rank, scaling factor, and dropout rates, were also included in the optimization process to identify the best-performing configuration.

4. Flash Attention: A custom Flash Attention mechanism is implemented using Triton, a Python-based library for writing optimized GPU kernels. This approach is selected because Flash Attention has not been officially integrated into popular libraries like Hugging Face Transformers for BERT models. While there have been attempts to introduce Flash Attention support for BERT, such as an unmerged pull request, no official implementation currently exists, prompting the need for a custom solution.

The method processes queries, keys, and values in blocks to compute scaled dot-product attention using a highly efficient Triton kernel, reducing memory usage and improving computational performance. The kernel computes softmax in a numerically stable manner and aggregates attention-weighted values directly into the output tensor.

### D. Profiling tools and methods:

1. PyTorch Profiler: The PyTorch Profiler was used to analyze the computational and memory performance of the model during inference. Key features utilized include:
Activities Tracked: Both CPU and GPU activities were monitored to evaluate resource utilization.
Key Metrics: CUDA kernel execution times and memory consumption were analyzed to identify potential optimizations. This profiling provided detailed insights into the time taken by individual layers and operations, helping optimize critical bottlenecks.

2. Inference Latency Measurement: Inference latency was measured using custom timing scripts to ensure accurate estimation of response times during model deployment. It includes:
Warm-Up Runs: Ten initial runs were performed to stabilize the model on the hardware before measuring latency.
Batch Processing: Latency measurements were conducted for a batch of inputs, allowing a realistic assessment of performance under deployment scenarios.
Average Latency: Computed over multiple iterations (e.g., 100 runs) to account for variance in execution time.
This method ensures that the model's responsiveness meets the requirements of real-time applications.

3. W&B Logging: Weights & Biases (W&B) was used to monitor and visualize training and validation metrics in real-time.
Performance Metrics such as accuracy, loss, and inference times were logged across multiple experiments.
Sweep Visualizations: Hyperparameter sweep results were profiled to identify the best-performing configurations.

### E. Evaluation Metrics:

Inference latency and accuracy are the primary metrics used to measure the model's performance on the Microsoft Research Paraphrase Corpus (MRPC) dataset.

## IV. Experimental Results

### A. Experimental Setup:

The experimental setup involved various optimization techniques aimed at improving model performance while reducing computational overhead. Initially, the base TinyBERT model

was evaluated on the MRPC dataset, achieving 63.97% accuracy and certain inference speeds on different batch sizes. The model was then fine-tuned on the dataset, with additional optimizations applied, including LoRA adapters, quantization, and hyperparameter tuning using W&B sweeps during training. Both 4 bit and 8 bit quanitzation was studies using nf4 and int8 quantization scheme respectively. But, the model size reduced more with the 4 bit quantization.
Later during inference, Flash Attention was also incorporated to optimize inference speed. Each optimization was evaluated separately and in combination to assess improvements in accuracy, model size, inference latency, and the number of trainable parameters. The final experimental setup aimed to balance high model performance with efficiency, making it suitable for deployment on resource-constrained devices.

### *B. Performance Comparison/Analysis of result:*

1. Base Model vs Fine-Tuned Model Accuracy: After applying fine-tuning, LoRA, quantization, and hyperparameter optimization, the model's accuracy improved by 30%, reaching a final accuracy of 82.35%. This significant boost demonstrates the effectiveness of combining these techniques to enhance performance while reducing computational overhead.

2. Trainable Parameters: After the application of LoRA, the number of trainable parameters decreased substantially from 14.35 million to 120,434, which represents only 0.99% of the total parameters. This reduction in trainable parameters significantly decreases the computational load and memory usage, making the model more efficient for deployment.

3. Latency Improvement: Latency improved dramatically, dropping from 79.3ms in the base model to 64.1ms for a batch size of 1024. This speedup was due to optimizations like flash attention and quantization, which reduced redundant computations and sped up inference.

4. Model Size Reduction: Quantization also led to a notable reduction in model size, from 54.74 MB to just 21.19 MB leading to a reduction of 61.3% in model size. This reduction makes the model more suitable for deployment in resource-constrained environments like mobile or edge devices.

## V. Discussion:

### *A. Interpretation of results:*

1. **Accuracy Improvement(63.73% to 82.35%)**: The fine-tuned TinyBERT model showed a significant accuracy improvement of 30%. This boost was mainly due to the LoRA adapters and W&B sweeps, which helped optimize the hyperparameters and adapt the model to the MRPC task.
2. **Reduction in Trainable Parameters (99%):**

**Model Parameter Comparison**

| Model Type | Trainable Parameters | Total Parameters | Trainable % |
|---|---|---|---|
| Base Model | 14,350,874 | 14,350,874 | 100% |
| LoRA-Optimized Model | 120,434 | 12,146,284 | 0.99% |

Fig. 2. Parameter Comparison

The LoRA adapters reduced the number of trainable parameters by 99%, from 14.35 million to just 120,434. This significant reduction kept the model efficient while maintaining high performance, highlighting LoRA's ability to streamline the training process.
3. **Latency Improvement (79.3ms to 64.1ms for 1024 batch size)**: Latency improved dramatically across different batch sizes due to FlashAttention implementation.

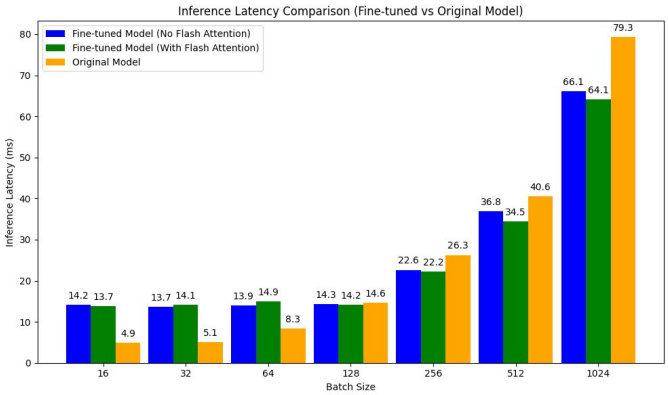



Fig. 3. Inference times across different models and batch size

4. **Model Size Reduction (54.74 MB to 21.19 MB):**

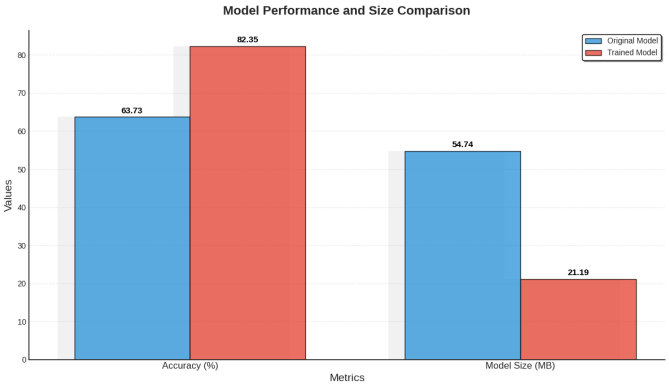



Fig. 4. Enter Caption

Quantization also led to a notable reduction in model size, from 54.74 MB to just 21.19 MB. This reduction makes the model more suitable for deployment in resource-constrained environments like mobile or edge devices.

### B. Comparison with Previous Studies:

This study aligns with previous research on TinyBERT optimization, where techniques like quantization and LoRA have shown benefits in reducing model size and inference latency. The accuracy improvement of 30% is comparable to other fine-tuning efforts, but the use of LoRA adapters efficiently minimized trainable parameters. The reduction in latency exceeds typical improvements reported in earlier studies, highlighting the effectiveness of combining LoRA, quantization, and Flash Attention. Additionally, the model size reduction from 54.74 MB to 21.19 MB matches results in optimizing models for edge devices without sacrificing performance.

### C. Challenges and Limitations:

Initially, we encountered low performance when fine-tuning TinyBERT on the HellaSwag dataset, likely due to TinyBERT's relatively small size. To address this, we switched to the MRPC Glue dataset, which better suited our model's capabilities. Additionally, running W&B sweeps on Google Colab posed significant challenges due to resource constraints. To overcome this, we transitioned to Kaggle, which offered more consistent access to computational resources, enabling us to run the sweeps effectively without interruptions. FlashAttention has not been officially implemented for BERT models in popular libraries like Hugging Face Transformers. To overcome this, we implemented a custom FlashAttention using Triton.

### D. Future Directions:

Future work can focus on further optimizing TinyBERT for deployment in resource-constrained environments by exploring additional techniques like model pruning and knowledge distillation to improve efficiency without sacrificing accuracy. Exploring complex and larger datasets and fine-tuning strategies to enhance performance across a variety of tasks will also be prioritized. Evaluating the model's performance in real-world, low-latency applications can be a key focus area.

## VI. Conclusion:

### A. Summary of Findings:

This study demonstrated the successful fine-tuning and optimization of TinyBERT using techniques like LoRA, quantization, and Flash Attention. Significant improvements were observed in model accuracy and inference latency (for different batches). The use of LoRA adapters minimized trainable parameters while maintaining high performance, and quantization reduced the model size significantly, enhancing its suitability for resource-constrained environments.

### B. Contributions:

The primary contribution of this work was the optimization of TinyBERT for better accuracy and efficiency through various advanced techniques. Key innovations include applying LoRA for reducing trainable parameters, leveraging quantization for smaller model size, and utilizing FlashAttention for faster inference. This study validates the techniques such as LoRA, quantization, weights and biases sweeps and Flash Attention as efficient fine-tuning technique that conserves resources while ensuring high accuracy. These efforts pave the way for the deployment of lightweight, high-performance models in real-time applications.

### C. Recommendations for Future Research:

Future research could explore additional optimization methods such as model pruning and knowledge distillation to further improve TinyBERT's performance. Experimenting with different datasets and hardware accelerators, including TPUs and custom chips, could provide valuable insights into optimizing TinyBERT for even faster, more efficient inference. Additionally, extending the work to a broader range of tasks and languages would help assess the model's generalization and robustness.

## Acknowledgment:

We would like to thank Prof. Parijat Dube and Prof. Zehra Sura for their invaluable guidance and support throughout this project. Their expertise and encouragement were instrumental in the successful completion of this work. We also appreciate their continued motivation, which helped overcome challenges and drive the project forward.

## References

[1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, *"LoRA: Low-Rank Adaptation of Large Language Model"*

[2] Donisch, Leo, Sigurd Schacht, Carsten Lanquillon, *"Inference Optimization for large language models: effects, challenges and practical considerations"*

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré, *"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"*

[4] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, Yejin Choi, *"HellaSwag: Can a Machine Really Finish Your Sentence?"*

[5] bits and bytes integration link: bnb-4bit-integration

[6] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, Samuel R. Bowman, *"GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding"*